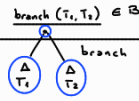
INDUZIONE STRUTTURALE

Definiamo BTree come

$$\begin{cases} \text{foglia} \in \text{Btree} & \bullet \text{ foglia} \\ T_1 \in \text{Btree} \ \text{e} \ T_2 \in \text{Btree} \Rightarrow \text{branch}(T_1, T_2) \in \text{BTree} & \end{cases}$$



Definiamo n: BTree $\to \mathbb{N}$ conta i nodi; d: un albero binario

$$\begin{cases} n(\text{foglia}) = 1 \\ n(\text{branch}(T_1, T_2)) = 1 + n(T_1) + n(T_2) \end{cases}$$

Definiamo h: BTree $\to \mathbb{N}$ calcola l'altezza dell'albero

$$\begin{cases} h(\text{foglia}) = 0 \\ h(\text{branch}(T_1, T_2)) = 1 + \max(h(T_1), h(T_2)) \end{cases}$$

(calcola il valore massimo tra i due valori)

Dimostrare per induzione strutturale che

(*) $\forall T \in \text{Btree}. \quad n(T) \leq 2^{h(T)+1} - 1$

Base: Dobbiamo dimostrare (*) per gli elementi foglia

T = foglia $\quad n(T) = n(\text{foglia}) = 1$
$h(T) = h(\text{foglia}) = 0$

Calcoliamo $2^{h(T)+1} - 1 = 2^{0+1} - 1 = 1$
$n(T) = 2^{h(T)+1} - 1$ ✓

Passo induttivo: $T = \text{branch}(T_1, T_2)$

Supponiamo per Ip. ind. che

Ip. indotte → (*) $n(T_1) \leq 2^{h(T_1)+1} - 1$
(*) $n(T_2) \leq 2^{h(T_2)+1} - 1$

dimostriamo che $n(T) < 2^{h(T)+1} - 1$

$n(T) = 1 + n(T_1) + n(T_2)$ $\quad h(T) = \max(h(T_1), h(T_2)) + 1$
$h = h$ $\quad h \geq h(T_1) + 1$, $h \geq h(T_2) + 1$

per def di n: $n(T) = 1 + n(T_1) + n(T_2) \leq 1 + 2^{h(T_1)+1} - 1 + 2^{h(T_2)+1} - 1$

per (*) e (*) Hp. ind.: $\leq 2^h + 2^h - 1$
$= 2^{h+1} - 1$
$= 2^{h(T)+1} - 1$

BTree, definiamo

ℓ: BTree $\to \mathbb{N}$ conta le foglie

$$\begin{cases} \ell(\text{foglia}) = 1 \\ \ell(\text{branch}(T_1, T_2)) = \ell(T_1) + \ell(T_2) \end{cases}$$

Dimostrare che $\ell(T) \leq 2^{h(T)} \quad \forall T \in \text{BTree}$

Base: T = foglia : $\ell(T) = 1 \quad 1 \leq 2^0$ ✓
$h(T) = 0$

Passo induttivo: $T = \text{branch}(T_1, T_2)$

Hp. ind.: $\ell(T_1) \leq 2^{h(T_1)} \quad \ell(T_2) \leq 2^{h(T_2)}$

Dobbiamo dimostrare che $\ell(T) \leq 2^{h(T)}$

$\ell(T) = \ell(T_1) + \ell(T_2)$ $\quad h(T) = 1 + \max(h(T_1), h(T_2))$
h

$\ell(T_1) + \ell(T_2) \leq 2^{h(T_1)} + 2^{h(T_2)}$ (Hp. ind)

$h = \max(h(T_1), h(T_2))$
$\Rightarrow h(T_1) \leq h$
$h(T_2) \leq h$

$\leq 2^h + 2^h = 2^{h+1} = 2^{h(T)}$

Exp (espressioni aritmetiche) definite induttivamente

Def di Exp
$$\begin{cases} n \in \mathbb{E}xp \\ e_1, e_2 \in \mathbb{E}xp. \quad e_1 + e_2 \in Exp \quad e_1 - e_2 \in Exp \end{cases}$$

Def val: Exp $\to \mathbb{N}$ numero di numeri presenti in una exp

$$\begin{cases} val(n) = 1 \\ val(e_1 + e_2) = val(e_1 - e_2) = val(e_1) + val(e_2) \end{cases}$$

Def op: Exp $\to \mathbb{N}$ numero di operatori in Exp $(+, -)$

$$\begin{cases} op(n) = 0 \\ op(e_1 + e_2) = op(e_1 - e_2) = op(e_1) + op(e_2) + 1 \end{cases}$$

Dimostrare che $val(e) = op(e) + 1 \quad \forall e \in Exp$

Base: $e = n$ $\quad val(e) = val(n) = 1$
$op(e) = op(n) = 0$ $\quad 1 = 0 + 1$ ✓

Passo ind.: $e_1 + e_2$ $(e_1 - e_2$ Analogo)

Hp. ind. $\quad val(e_1) = op(e_1) + 1$
$val(e_2) = op(e_2) + 1$

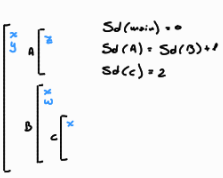
Dobbiamo dim. che $val(e) = op(e) + 1$

$val(e) = val(e_1) + val(e_2) = op(e_1) + 1 + op(e_2) + 1$ (Hp. ind)

$= op(e_1) + op(e_2) + 1 + 1 = op(e) + 1$

$op(e)$ per def di op

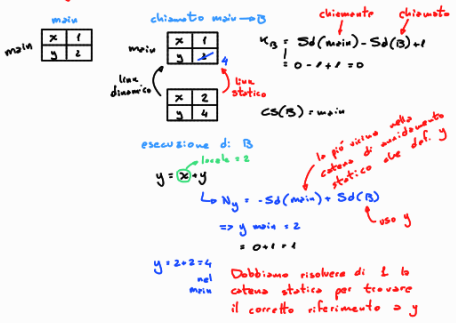
```
{int x=1;
 int y=2;
 void A(){ int z = x+y;}
 void B(){ int x = 2;
           int w=4;
           void C(){ int x = y+w;
                     A();
                   }
           y = x+y;
           C();
         }
 B();
}
```



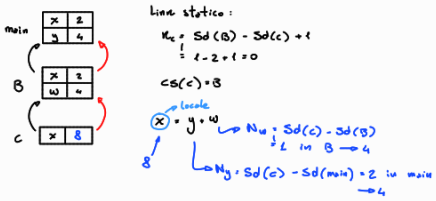
Sd(main) = 0
Sd(A) = Sd(B)+1
Sd(C) = 2

main → B → C → A

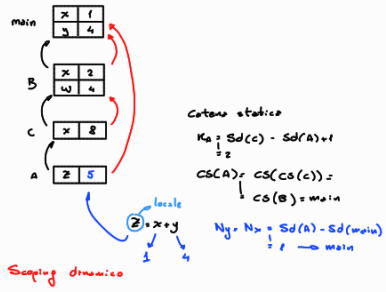
Scoping statico



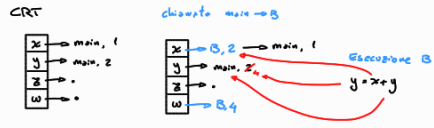
Chiamata B→C



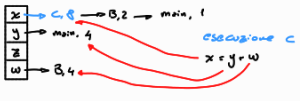
Chiamata C→A



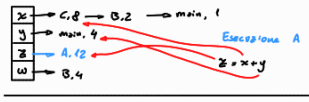
Scoping dinamico



Chiamata B→C



Chiamata C→A



Ricorsione

```
liste function (liste list){
    if (lenght (list) =0) then return list = ()
    else return concat (function (cdr(list)), car(list))
   }
```

function (3,6,9) (9,6,3)
↳ concat (function (6,9), 3) (9,6)
↳ concat (function (9), 6) (9)
↳ concat (function (()), 9)
b
()

La funzione inverte gli elementi della lista

```
liste functionre (liste list, liste res){
    if lenght (list) =0 return res;
    else return functionre (cdr(list), concat (car(list), res));
   }
liste function (liste list) { return functionre(list, ()); }
```

function (3,6,9) (9,6,3)
↳ functionre ((3 6 9), ()) (1)
↳ functionre ((6 9), concat (3, ())) (3)
↳ functionre ((9), concat (6, (3))) (6 3)
↳ function (()), concat (9, (6 3))) (9 6 3)